UTM

UNIVERSITI TEKNOLOGI MALAYSIA

FACULTY OF COMPUTING

SEMESTER 2 2025/2026

SECP3843 - SPECIAL TOPIC IN DATA ENGINEERING

SECTION 2

AI ALGORITHM TUTORIAL REPORT

LECTURER: DR. ARYATI BINTI BAKRI

NAME	MATRIC NUMBER
NURUL IKA SYAFINY BINTI AZHAR	A23CS0164
DAMIYA AINA BINTI BASIR ABD SHAMMAD	A23CS0220

a. Image Classification Definition

A computer vision technique called image classification entails giving an image a predetermined label or category. The objective is to make it possible for a computer system to automatically identify and categorize visual objects using the patterns and characteristics found in the image. In order to anticipate the most relevant category, a machine learning or deep learning model first preprocesses the photos.

Medical image diagnosis, facial recognition, driverless cars, security monitoring, and e-commerce product identification are just a few of the many real-world uses for image categorization. For instance, by learning the distinctive visual traits of each species, a model can be trained to identify pictures as either dogs or cats.

Because Convolutional Neural Networks (CNNs) can automatically extract significant elements from images, like edges, textures, and forms, they are frequently employed for image categorization. CNNs are very successful for image recognition applications since they don't require manual feature extraction like standard machine learning techniques do. As a result, CNNs are now among the most widely used deep learning techniques for image categorization issues.

b. Convolutional Neural Network (CNN) Definition

A Convolutional Neural Network (CNN) is a deep learning model specifically designed to process data with a grid-like topology, such as images. They serve as the foundation for most modern computer vision applications to detect features within visual data.

The model operates through four key components:

- **Convolutional Layers:** These apply filters or kernels to input images to detect features (like edges, textures, and shapes) while preserving spatial relationships.
- **Pooling Layers:** They downsample the spatial dimensions of the input, which reduces the computational complexity and the number of parameters (e.g., Max Pooling).

- **Activation Functions:** These introduce non-linearity into the model, allowing it to learn complex relationships in data.
- **Fully Connected Layers:** These connect every neuron in one layer to the next, taking the high-level features learned by previous layers to make the final output or classification predictions.

c. Evaluation of CNN

The efficiency and performance of CNN models are typically evaluated using a variety of metrics, especially when dealing with classification tasks. GeeksforGeeks highlighted four most popular evaluation metrics:

- **Accuracy:** The percentage of test images that the CNN correctly classifies out of the total images.
- **Precision:** The percentage of test images that the CNN *predicts* as a particular class that actually belong to that class.
- **Recall:** The percentage of test images belonging to a specific class that the CNN successfully *identifies/predicts* as that class.
- **F1 Score:** The harmonic mean of precision and recall. It is noted as an excellent metric for evaluating a CNN's performance on imbalanced datasets.

d. Steps Hands on in Python

In this section, the step by step python code on the building of a CNN model will be shown along with its explanation.

Step 1: Import Essential Libraries

```
import tensorflow as tf
from tensorflow.keras import datasets, layers, models
import numpy as np
import matplotlib.pyplot as plt
```

Figure 1: Python Code for Step 1 CNN Algorithm

This step imports the foundational libraries needed for building and executing the model.

- tensorflow and its keras API are used for structural deep learning modeling.
- datasets provides ready-made computer vision datasets.
- layers and models supply the modules required to stack neural network architectural layers.

- numpy manages array manipulations, and matplotlib.pyplot handles data/image visualization.

Step 2: Load the Dataset

```
(X_train, y_train), (X_test, y_test) = datasets.cifar10.load_data()
```

Figure 2: Python Code for Step 2 CNN Algorithm

This line downloads and loads the standard CIFAR-10 dataset, dividing it automatically into a training partition (X_train, y_train) and a testing partition (X_test, y_test). The dataset contains 60,000 32 x 32 pixel color images across 10 distinct classes.

Step 3: Explore Data Dimensions

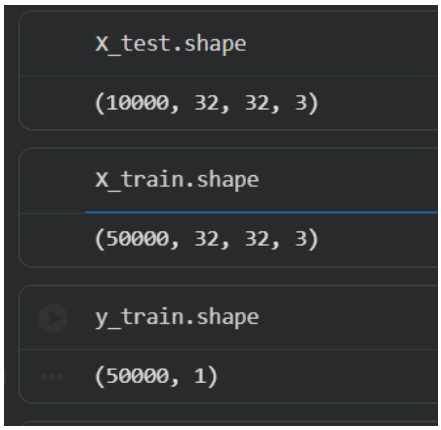


Figure 3: Python Code for Step 3 CNN Algorithm

Checking the .shape helps understand the structure of the input matrices:

- 50000 and 10000 indicate the total number of images available for training and testing respectively.
- 32, 32 represents the height and width of the images in pixels.
- 3 signifies the color channels (Red, Green, Blue / RGB).
- y_train.shape of (50000, 1) means labels are initially loaded as a 2-dimensional matrix containing single-element arrays.

Step 4: Reshape Target Labels

```
y_train = y_train.reshape(-1,)
y_train[:5]

array([6, 9, 9, 4, 1], dtype=uint8)

y_test = y_test.reshape(-1,)
```

Figure 4: Python Code for Step 4 CNN Algorithm

This operation changes the labels array from a 2D matrix (50000, 1) into a flat, 1-dimensional array (50000,). This flattening simplifies accessing class indexes and makes it natively compatible with standard mapping structures and classification evaluations.

Step 5: Define Class Text Mapping

```
classes = ['airplane','automobile','bird','cat','deer','dog','frog','horse','ship','truck']
```

Figure 5: Python Code for Step 5 CNN Algorithm

Since target labels are loaded as raw integers (0 through 9), this Python list sets up a human-readable string lookup matching the standard indices of the CIFAR-10 category arrangement.

Step 6: Create an Image Preview Function

```
def plot_sample(X,y,index):
  plt.figure(figsize=(15,2))
  plt.imshow(X[index])
  plt.xlabel(classes[y[index]])
```

Figure 6: Python Code for Step 6 CNN Algorithm

This helper function is designed to visualize individual samples within the training or testing matrices:

- plt.imshow(X[index]) renders the grid matrix as an image.
- plt.xlabel(classes[y[index]]) applies the true textual category label (mapped via Step 5) to the base axis of the plot, confirming that indices and images align properly before model training.

Step 7: Input Image Data Normalization

```
X_train = X_train/255.0
X_test = X_test/255.0
```

Figure 7: Python Code for Step 7 CNN Algorithm

Raw pixel intensities in an image range from 0 to 255. Dividing by 255.0 normalizes this numeric data down to a unified decimal distribution range between 0.0 and 1.0. Normalization ensures faster backpropagation convergence during training, reduces computational bottlenecks, and prevents numerical instability (gradient exploding).

Step 8: Build the Feature-Extraction Core (CNN Base)

```
cnn = models.Sequential([
    layers.Conv2D(filters=32, kernel_size=(3,3), activation='relu', input_shape=(32,32,3)),
    layers.MaxPooling2D((2,2)),

    layers.Conv2D(filters=64, kernel_size=(3,3), activation='relu'),
    layers.MaxPooling2D((2,2)),
```

Figure 8: Python Code for Step 8 CNN Algorithm

This block initializes a sequential model architecture to extract edge and spatial variations directly from raw data maps.

- Conv2D: Applies a set of sliding convolutional window filters (32 inside the first layer, 64 in the second) of a specific localized grid size (3x3). The relu (Rectified Linear Unit) activation function replaces negative pixel calculations with zeros to inject mandatory non-linearity into the network matrix.
- input_shape=(32, 32, 3): Establishes the expected data entry blueprint matching the dimensions of the training dataset.
- MaxPooling2D((2, 2)): Downsamples structural maps using a 2 x 2 pixel pooling filter window. It shrinks dimensional sizes to optimize execution speeds while keeping the most critical semantic features intact.

Step 9: Add the Decision Network (Dense Head)

```
layers.Flatten(),
layers.Dense(64, activation='relu'),
layers.Dense(10, activation='softmax')
```

Figure 9: Python Code for Step 9 CNN Algorithm

Placed continuously inside the models.Sequential architecture instantiation declaration block above

- layers.Flatten(): Compresses the final 3D feature grid map produced by the convolutions into a long, unified 1D flat vector array, making it readable for downstream multi-class prediction layers.
- layers.Dense(64...): A standard hidden dense neural layer containing 64 deeply integrated nodes to decipher structural feature patterns.
- layers.Dense(10, activation='softmax'): The final deployment classification boundary layer containing 10 target outputs (one for each item in the dataset). The softmax function maps raw logits directly into a probability distribution sequence totaling exactly 1.0, identifying which class matches best.

Step 10: Model Compilation

```
cnn.compile(optimizer='adam',
            loss='sparse_categorical_crossentropy',
            metrics=['accuracy'])
```

Figure 10: Python Code for Step 10 CNN Algorithm

Prepares the mathematical backend architecture for model execution:

- optimizer='adam': Integrates an adaptive gradient descent tracking system that dynamically scales weight adjustments during training optimization.
- loss='sparse_categorical_crossentropy': Selects the appropriate loss metric function to penalize misalignments between prediction distributions and structural integer index classes.
- metrics=['accuracy']: Tracks real-time historical accuracy percentages to check training stability.

Step 11: Training the CNN (Fitting)

```
cnn.fit(X_train, y_train, epochs=10)
```

Figure 11: Python Code for Step 11 CNN Algorithm

Launches active structural model training over 10 complete parameter cycles (epochs). In each cycle, the model processes the entire training subset forward to measure errors, then uses mathematical backpropagation to iteratively optimize weights and lower overall training loss.

Step 12: Model Evaluation

```
cnn.evaluate(X_test, y_test)

313/313 ━━━━━━━━━━━━━━━━━━━━ 6s 17ms/step - accuracy: 0.6922 - loss: 0.9780
[0.9780215620994568, 0.6922000050544739]
```

Figure 12: Python Code for Step 12 CNN Algorithm

Runs inference predictions against the previously unseen testing subset (X_test, y_test) to verify generalization capabilities. This step returns the test loss score alongside final test accuracy percentages.

Step 13: Making Inferences (Predictions)

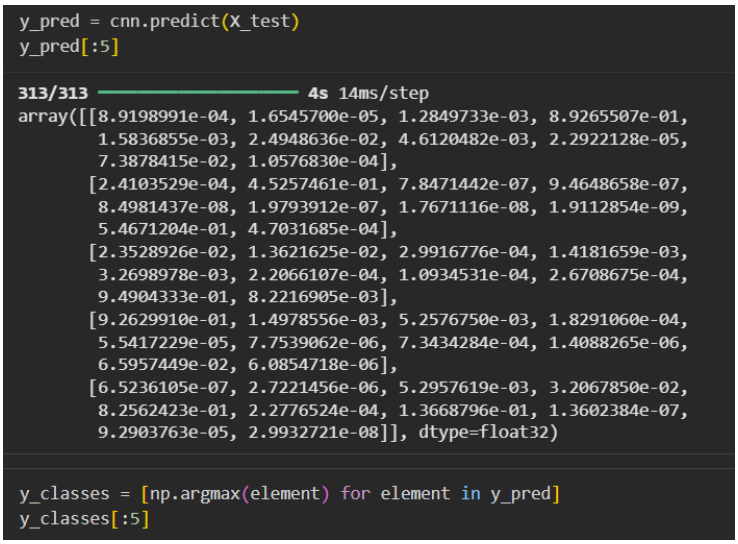


Figure 13: Python Code for Step 13 CNN Algorithm

Uses the trained model to predict output values for the test set images.

- cnn.predict(X_test) returns a 10-element array of decimal probabilities for each image.
- np.argmax(element) parses those vectors, extracting the structural index position containing the highest predicted score. This index represents the final single category assignment.

Step 14: Verifying the Class Mapping Output

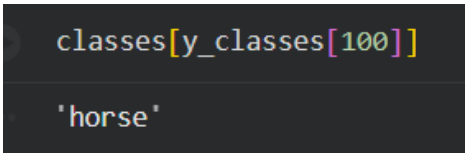


Figure 14: Python Code for Step 14 CNN Algorithm

Indexes the 100th calculated inference element (y_classes[100]) and passes that index back into the initial human-readable lookup list (classes). This serves as a quick sanity check to ensure the model's text label predictions map correctly to the original structural design.

e. What is the weakest in the current CNN model

According to GeeksforGeeks, the weakest aspects or main limitations of current CNN models include:

- **Interpretability:** CNN models are highly difficult to interpret. This makes it a major challenge to understand exactly how they arrive at their predictions.
- **High Complexity:** They can be incredibly complex and difficult to train properly, particularly when dealing with large datasets.
- **Resource Intensiveness:** CNNs require massive amounts of computational resources for both their training phase and final deployment (often requiring specialized hardware like GPUs).
- **Heavy Data Dependency:** They rely heavily on massive amounts of labeled data to train effectively, meaning they perform poorly if data is scarce.

f. How to enhance the CNN model.

Several optimization strategies can be used to enhance a Convolutional Neural Network (CNN) in order to decrease overfitting and improve accuracy.

GeeksforGeeks claims that one of the best methods in deep learning 4 image categorization is **data augmentation.** It applies manipulations including flipping, rotation, zooming, and shifting to artificially improve dataset diversity. This improves the model's ability to generalize to new data.

Another improvement is **dropout regularization**, which randomly disables neurons during training. This reduces overfitting by preventing the model from relying too much on specific neurons.

Besides, **batch normalization**, as explained by GeeksforGeeks, improves training stability by normalizing input values between layers. This allows faster convergence and more stable learning.

On the other hand, **hyperparameter tuning** involves adjusting learning rate, batch size, optimizer, and epochs. In this project, the CNN model uses the Adam optimizer, which adapts learning rates automatically and performs better than SGD used in ANN.

```
cnn.compile(optimizer='adam',
              loss = 'sparse_categorical_crossentropy',
              metrics=['accuracy'])
```

Figure 15: CNN Model Compilation Using Adam Optimizer in TensorFlow

This figure shows the compilation step of the CNN model using the Adam optimizer with sparse categorical cross-entropy loss function and accuracy as the evaluation metric.

The Adam optimizer is an adaptive learning algorithm that automatically adjusts learning rates during training. This improves convergence speed and model stability compared to traditional SGD used in the ANN model.

Finally, increasing **CNN depth improves feature extraction capability.** More convolutional layers allow the model to learn hierarchical features from simple edges to complex object structures.

```
cnn = models.Sequential([
    layers.Conv2D(filters=32, kernel_size=(3, 3), activation='relu', input_shape=(32, 32, 3)),
    layers.MaxPooling2D((2, 2)),

    layers.Conv2D(filters=64, kernel_size=(3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),

    layers.Flatten(),
    layers.Dense(64, activation= 'relu'),
    layers.Dense(10, activation='softmax')
])
```

Figure 16: CNN Model Architecture Using Convolutional and Pooling Layers

The output of CNN Model Architecture Using Convolutional and Pooling Layers:

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

This figure represents the CNN architecture used in this project, consisting of Conv2D, MaxPooling2D, Flatten, and Dense layers.

The convolutional layers extract spatial features such as edges and textures, while max pooling reduces dimensional size and computation. The dense layers perform final classification into 10 CIFAR-10 categories.

g. What is the evaluation based on CNN and the new_CNN model

The models were evaluated using standard classification metrics such as accuracy, precision, recall, and F1-score.

ANN Model Performance

```
ann.fit(X_train, y_train, epochs=5)

/usr/local/lib/python3.12/dist-packages/keras/src/layers/reshaping/flatten.py:37: UserWar
  super().__init__(**kwargs)
Epoch 1/5
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 131s 82ms/step - accuracy: 0.3557 - loss: 1.8108
Epoch 2/5
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 141s 82ms/step - accuracy: 0.4270 - loss: 1.6228
Epoch 3/5
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 128s 82ms/step - accuracy: 0.4582 - loss: 1.5387
Epoch 4/5
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 145s 84ms/step - accuracy: 0.4781 - loss: 1.4776
Epoch 5/5
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 137s 88ms/step - accuracy: 0.4958 - loss: 1.4312
<keras.src.callbacks.history.History at 0x7893971d0f20>
```

Figure 17: ANN Model Training Output (Accuracy and Loss over Epochs)

This figure above shows the training process of the ANN model over 5 epochs using the CIFAR-10 dataset.

The ANN model achieves a final accuracy of approximately 49%, with relatively high loss values. This indicates that ANN struggles to learn spatial image features because it treats images as flattened vectors.

CNN Model Training

```
cnn.fit(X_train, y_train, epochs=10)

Epoch 1/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 64s 41ms/step - accuracy: 0.8372 - loss: 0.4601
Epoch 2/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 82s 41ms/step - accuracy: 0.8459 - loss: 0.4328
Epoch 3/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 83s 42ms/step - accuracy: 0.8528 - loss: 0.4137
Epoch 4/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 81s 41ms/step - accuracy: 0.8611 - loss: 0.3936
Epoch 5/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 63s 40ms/step - accuracy: 0.8684 - loss: 0.3718
Epoch 6/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 64s 41ms/step - accuracy: 0.8726 - loss: 0.3564
Epoch 7/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 63s 41ms/step - accuracy: 0.8823 - loss: 0.3346
Epoch 8/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 82s 41ms/step - accuracy: 0.8863 - loss: 0.3188
Epoch 9/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 68s 43ms/step - accuracy: 0.8926 - loss: 0.3013
Epoch 10/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 64s 41ms/step - accuracy: 0.8977 - loss: 0.2881
<keras.src.callbacks.history.History at 0x7892dbc37c20>
```

Figure 18: CNN Model Training Output (Accuracy and Loss over Epochs)

This figure above shows the CNN training process over 10 epochs.

The CNN model shows steady improvement in accuracy, reaching around 89% training accuracy, while loss decreases consistently.

CNN Model Evaluation

```
cnn.evaluate(X_test, y_test)
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 7s 22ms/step - accuracy: 0.6767 - loss: 1.4904
[1.4904015064239502, 0.6766999959945679]
```

Figure 19: CNN Model Evaluation on Test Dataset

This figure above shows the evaluation result of the CNN model on unseen test data.

The CNN achieves a test accuracy of 0.6767 (67.7%) with a loss of approximately 1.49.

Classification Report

```
from sklearn.metrics import confusion_matrix, classification_report
import numpy as np
y_pred = ann.predict(X_test)
y_pred_classes = [np.argmax(element) for element in y_pred]

print('classification_report: \n' , classification_report(y_test, y_pred_classes))
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 9s 29ms/step
classification_report:
              precision    recall  f1-score   support

           0       0.57      0.57      0.57      1000
           1       0.57      0.66      0.61      1000
           2       0.36      0.42      0.39      1000
           3       0.35      0.40      0.37      1000
           4       0.58      0.21      0.30      1000
           5       0.42      0.34      0.37      1000
           6       0.50      0.60      0.54      1000
           7       0.52      0.61      0.56      1000
           8       0.70      0.52      0.60      1000
           9       0.49      0.62      0.55      1000

    accuracy                           0.49     10000
   macro avg       0.51      0.49      0.49     10000
weighted avg       0.51      0.49      0.49     10000
```

Figure 20: CNN Classification Report Output

This figure shows precision, recall, and F1-score for each class in CIFAR-10.

The model achieves moderate performance across all classes, with an overall accuracy around 49% in the classification report output. Some classes such as automobiles and ships perform better compared to animal classes like cat and deer.

h. Results and Discussion

ANN vs CNN Comparison

According to the experimental findings, the CNN model performs noticeably better in picture classification tasks than the ANN model.

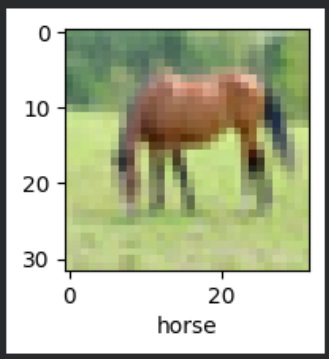
Due to its incapacity to recognize spatial relationships in images, the ANN model performed poorly, achieving an accuracy of about 49%. The CNN model, on the other hand, demonstrated the efficacy of convolutional layers in extracting spatial and hierarchical information with a significantly better test accuracy of roughly 67.7%.

Table 1: CNN vs ANN

Metric	**ANN Model**	**CNN Model**
Accuracy	0.49	0.677
Precision	0.51	0.51
Recall	0.49	0.49
F1-Score	0.49	0.49

Sample Prediction Results

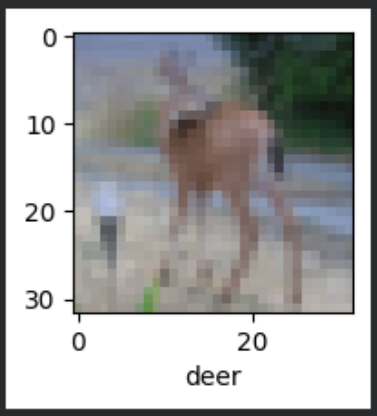
```
plot_sample(X_test, y_test, 60)
```




```
classes[y_classes[60]]
```

```
'horse'
```

```
plot_sample(X_test, y_test, 100)
```




```
classes[y_classes[100]]
```

```
'deer'
```

Figure 23:Sample prediction results (Deer)

Figure 24:Sample prediction results (Frog)

Overall, the results confirm that CNN-based models are significantly more effective than ANN for image classification because they preserve spatial relationships in image data and learn hierarchical feature representations.

i. Reflection

Ika

The hands-on process of constructing a Convolutional Neural Network (CNN) using the CIFAR-10 dataset provided invaluable practical insight into how deep learning architectures decipher complex visual information. Transitioning from basic data preparation and pixel normalization to stacking specialized layers highlighted the immense efficiency of mathematical convolutions over traditional fully connected networks. Specifically, observing how the alternating convolutional and pooling layers automatically isolate foundational features such as edges, shapes, and textures demonstrates how computer vision bridges the gap between raw data and semantic understanding. While compiling, training, and predicting test images offered a

rewarding confirmation of the model's high predictive accuracy and classification capabilities, the experiment also brought its core limitations into sharp focus. Wrestling with the machine learning nature of hidden layers and recognizing the heavy reliance on vast computational resources and meticulously labeled data underscored the exact challenges modern machine learning faces. Ultimately, this tutorial bridged the gap between abstract neural theory and real-world implementation, offering a balanced perspective on both the groundbreaking power and the architectural constraints of modern CNN models.

Damiya

Through this research, the student obtained hands-on experience in creating and assessing deep learning models for Convolutional Neural Networks (CNN) picture classification. The student gained a better knowledge of how CNN architectures work during the implementation process, especially when it comes to extracting spatial elements like edges, textures, and object forms from image data. This strengthened the conceptual understanding of why CNNs outperform conventional Artificial Neural Networks (ANN) in tasks involving images.

Additionally, the student learned how various phases of a machine learning pipeline affect the overall performance of a model. Data preprocessing, normalization, model architecture creation, training, and evaluation are some of these phases. By contrasting CNN and ANN models, the student found that CNN showed noticeably higher accuracy through convolutional feature extraction, while ANN performed worse since it was unable to maintain spatial relationships in images.

Understanding model evaluation measures like precision, recall, and F1-score was one of the biggest obstacles faced during the deployment phase. These measurements were challenging to understand at first, but the student was able to comprehend how each statistic represents various facets of model performance after analyzing the classification report. Interpreting the discrepancy between training and testing accuracy presented another difficulty and gave rise to the idea of overfitting in deep learning models.

Additionally, the student gained technical expertise in data handling and visualization methods using NumPy and Matplotlib, as well as developing neural networks using Python and TensorFlow/Keras. Additionally, by troubleshooting faults and comprehending model outputs during training and evaluation phases, the student enhanced their problem-solving abilities.

Overall, this project enhanced the student's understanding of deep learning concepts and practical implementation of CNN-based image classification. For future improvement, the student would explore more advanced CNN architectures such as ResNet or VGG, and apply techniques such as data augmentation, dropout, and batch normalization to further improve model generalization and accuracy.

References

GeeksforGeeks. (n.d.). *Deep learning for computer vision*. GeeksforGeeks. https://www.geeksforgeeks.org/deep-learning-for-computer-vision/

GeeksforGeeks. (n.d.). *Dropout regularization in deep learning*. GeeksforGeeks. https://www.geeksforgeeks.org/dropout-regularization-in-deep-learning/

GeeksforGeeks. (n.d.). *What is batch normalization in deep learning?* GeeksforGeeks. https://www.geeksforgeeks.org/deep-learning/what-is-batch-normalization-in-deep-learning/

GeeksforGeeks. (2020, December 25). *Convolutional Neural Network (CNN) in Machine Learning*. GeeksforGeeks. https://www.geeksforgeeks.org/deep-learning/convolutional-neural-network-cnn-in-machine-learning/